

Atividade Prática de Compiladores

Análise Léxica

Objetivo

Implementar um **analisador léxico**, ou seja, um programa que lê um texto de entrada e produz a sequência de **tokens** correspondente.

Em ambas as partes, o analisador deve reconhecer:

- palavras reservadas;
- identificadores;
- números inteiros;
- números reais;
- operadores;
- delimitadores;
- strings;
- comentários;
- espaços em branco e quebras de linha;
- erros léxicos.

Para cada token reconhecido, informe o tipo, o lexema e a linha:

```
LINHA 2: KEYWORD(inteiro)
LINHA 2: IDENTIFIER(idade)
LINHA 2: DOLLAR($)
LINHA 3: IDENTIFIER(idade)
LINHA 3: ASSIGN(=)
LINHA 3: INTEGER(25)
```

Os nomes dos tipos de token são escolha sua, desde que cada token fique claramente identificado.

1 Parte 1. Linguagem de sua escolha

Escolha uma linguagem que você já conheça (C, C++, C#, Java, JavaScript, Python, Lua, Kotlin, Swift, Rust, Go, ou outra combinada com o professor) e escreva um analisador léxico para ela.

Aqui você tem liberdade para definir o conjunto de tokens e organizar a implementação como preferir.

2 Parte 2. MiniLang

A **MiniLang** é uma linguagem fictícia. A especificação dela são os cinco programas a seguir: leia o código, deduza as regras léxicas e implemente o analisador. Todos os alunos entregam o analisador da mesma linguagem, então não altere as regras que você observar nos programas.

A única coisa que o código não mostra é o conjunto completo de símbolos. Ele é este, e nada fora dele pertence à linguagem:

Aritméticos	+	-	*	/	%		
Relacionais	==	!=	<	>	<=	>=	
Lógicos	&&		!				
Atribuição	=						
Delimitadores	(	)	[	]	{	}	; \$

Palavras reservadas:

inicio	fim	inteiro	real	texto
booleano	se	faz	senao	repita_enquanto
para	de	ate	faca	leia
mostre	sim	nao	retorne	funcao

Quando dois tokens começam com o mesmo prefixo, vale o maior casamento possível. Isso decide `>=` contra `>`, e decide também os três casos que valem a pena olhar duas vezes nos programas abaixo: a chave, o número real e o `#`.

2.1 Programa 1. Cadastro e cálculo de salário

```
inicio

    inteiro idade$
    inteiro dependentes$
    real salario$
    real bonus$
    real salario_final$
    texto nome$

    nome = 'Maria'$
    idade = 35$
    dependentes = 2$

    salario = 4500,00$
    bonus = salario * 0,10$
    salario_final = salario + bonus$

    mostre('Nome: ')$
    mostre(nome)$

    mostre('Idade: ')$
    mostre(idade)$

    mostre('Dependentes: ')$
    mostre(dependentes)$

    mostre('Salario final: ')$
    mostre(salario_final)$

    #coment calculo adicional para dependentes
    salario_final = salario_final + (dependentes * 150,00)$

    mostre('Salario com beneficio: ')$
    mostre(salario_final)$

fim
```

2.2 Programa 2. Classificação de estudante

```
inicio

    texto nome$
    real nota1$
    real nota2$
    real nota3$
    real media$
    inteiro faltas$
    booleano aprovado$
```

```
nome = 'Carlos'$

nota1 = 8,5$
nota2 = 7,0$
nota3 = 9,0$

faltas = 3$

media = (nota1 + nota2 + nota3) / 3,0$

aprovado = nao$

se [media >= 7,0 && faltas <= 10] faz {

    aprovado = sim$

    mostre('Aluno aprovado.>')$
    mostre(nome)$
    mostre(media)$

} senao {

    aprovado = nao$

    mostre('Aluno reprovado.>')$
    mostre(nome)$
    mostre(media)$
}

se [media == 10,0 || faltas == 0] faz {
    mostre('Excelente resultado!')$
}

se [!aprovado && media > 0,0] faz {
    mostre('Verificar recuperacao.>')$
}

fim
```

2.3 Programa 3. Controle de estoque

```
inicio

inteiro estoque$
inteiro quantidade$
inteiro limite$
inteiro contador$
inteiro i$
real preco$
real total$
texto produto$

produto = 'Teclado'$
estoque = 25$
quantidade = 3$
limite = 10$
contador = 0$
preco = 149,90$
total = 0,0$

{---
  Inicio do processamento do estoque.
  O sistema verifica a quantidade disponivel
  e calcula o valor da compra.
---}

se [estoque >= quantidade] faz {

  estoque = estoque - quantidade$
  total = quantidade * preco$

  mostre('Produto: ')$
  mostre(produto)$

  mostre('Valor total: ')$
  mostre(total)$

} senao {
  mostre('Estoque insuficiente.')
```

```
}

se [estoque % 2 == 0] faz {
    mostre('Quantidade par em estoque.')
```

\$
}

#coment verificacao final
se [estoque <= limite && estoque != quantidade] faz {
 mostre('Atencao: estoque baixo.')\$
}

fim

2.4 Programa 4. Funções e leitura de dados

```
inicio

    funcao real media(real a; real b) {
        retorne (a + b) / 2,0$
    }

    funcao booleano passou(real m) {
        se [m >= 6,0] faz {
            retorne sim$
        }
        retorne nao$
    }

    real p1$
    real p2$
    real m$
    booleano ok$

    mostre('Digite a primeira nota: ')$
    leia(p1)$

    mostre('Digite a segunda nota: ')$
    leia(p2)$

    m = media(p1; p2)$
    ok = passou(m)$

    mostre('Media: ')$
    mostre(m)$

    ok = passou(7,5)$

    se [ok == sim] faz {
        mostre('Situacao: aprovado')$
    } senao {
        mostre('Situacao: reprovado')$
    }

fim
```

2.5 Programa 5. Entrada com erros léxicos

Toda linha deste programa contém pelo menos um erro. O analisador deve reportar cada um deles, com a linha, e continuar a leitura em vez de parar no primeiro.

```
inicio
    inteiro 123nome$
    real x$
    x = 10,25,30$
```

```
idade = 20 @ 5$
preco = 3.14$
total = preco , 2$
mostre('fim do calculo')$ #
texto s$
s = "aspas duplas nao existem aqui"$
texto t$
t = 'esta string nao termina$
{---
    comentario sem fechamento
fim
```

3 Testes adicionais

Crie pelo menos **cinco** entradas próprias, pequenas, cobrindo:

1. identificadores válidos, incluindo pelo menos um parecido com palavra reservada (**repita**, **faca2**, **naoSei**);
2. números inteiros e reais na mesma expressão;
3. os operadores compostos **<=**, **>=**, **==**, **!=**, **&&**, **||**;
4. comentários das duas formas, com uma chave **{** usada como delimitador na mesma entrada;
5. erros léxicos, pelo menos um diferente dos que aparecem no Programa 5.

4 Entrega

1. código-fonte do analisador da Parte 1;
2. código-fonte do analisador da Parte 2;
3. arquivos de entrada usados nos testes;
4. saída produzida para os cinco programas da Parte 2;
5. os cinco testes adicionais, com a saída de cada um.